Hybrid Formal Verification of Floating Point Designs, Revisited

Ashraf Alam, Louis-Emile Ploix, and Thomas Melham
Department of Computer Science
University of Oxford
Oxford, United Kingdom

***Abstract*—Proving the absolute correctness of a floating-point hardware arithmetic design requires proving compliance with the IEEE-754 floating-point standard. Bit-level model checking cannot handle the real-number mathematics involved, but interactive theorem provers (ITPs), such as Lean 4, are highly effective at verifying mathematical proofs of the type required. But proving the correctness of a complex hardware design from the RTL up typically demands high levels of human effort. Tackling the problem with a hybrid verification methodology that combines model checking and ITP is a well-known approach, but still involves significant human effort at the ITP level. In this work, we demonstrate a modernized methodology in which AI is staged to complete the mathematical level of proof. We illustrate our approach by proving the correctness of a floating-point square root hardware design.**

I. Introduction

As demonstrated by the infamous 1994 Intel Pentium FDIV bug, floating-point units can have subtle corner-case bugs. It is therefore important to verify complex floating-point designs, and the gold standard is full formal verification against the IEEE-754 specification [4]. For operations such as division or square root, non-linear expressions involving multiplication must be handled, and for IEEE-754 conformance, mathematical reasoning about real-number arithmetic is also required. But model checking (and in particular SAT solving) suffers from a significant explosion in problem scale and complexity when handling non-linear expressions at the bit level, and it cannot, of course, reason about the reals.

An old idea to address these issues is to use a hybrid verification methodology that combines BDD- or SAT-based symbolic simulation or model checking of low-level RTL with interactive theorem proving (ITP) for high-level mathematical IEEE-754 compliance [5, 1, 13, 14]. A typical methodology is to verify the RTL for the target floating point operation against a bit-level reference model, and then move over to ITP to show that the latter conforms to IEEE-754. Another approach is to conduct the proof entirely within a theorem prover that has a rich theory of bit vectors, alongside the reals. Russinoff's benchmark work with ACL2 is a notable example [11].

Although feasible in principle, these approaches can require substantial mathematical expertise and manual effort at the ITP level. Recent advances in artificial intelligence, particularly language-model-guided theorem proving, have produced systems that generate formal mathematical proofs and interact with proof assistants such as Lean to validate candidate proof steps [16, 15]. This raises the possibility of reducing the manual effort required for the mathematical phase of hybrid hardware verification.

In this paper, we develop and articulate a practical methodology that leverages this technology to enable much less laborous hybrid formal verification of floating point designs. We explain the methodology through a concrete example: the formal verification of the C910 open-source floating point SRT square root unit in FPNew [6] [2].

Our approach proceeds through three main phases:

- Invariant discovery
- High-level proof conceptualization and lemma verification
- Transfer to an ITP and formal proof completion

The process of invariant discovery is, of course, dependent on which floating-point arithmetic algorithm is implemented by the hardware, as well as any microarchitectural optimizations. In our example, the design fairly straightforwardly implements a radix-16 SRT square root algorithm. Key invariants are established and proven through an iterative process of refining SystemVerilog Assertions (SVA) in light of counterexamples provided by the model checker, a workflow very familiar to formal verification engineers.

Once key invariants are established as SVA, we conceptualize a high-level correctness proof. Our goal is to write and prove a set of assertions that together imply correctness according to everyday mathematical intuition. The proof itself need not be fully carried out. Guided by this conceptual proof structure, we then identify and prove specific, targeted lemmas on the RTL by further model checking.

Once we have confidence that our properties together imply correctness, we translate our SVA into the language of the ITP of choice—in our case, Lean 4. We then write a top-level correctness property in accordance with the IEEE-754 standard and attempt to prove this using the translated SVA as a set of assumptions. To address the manual overhead of proof using an ITP, we employ an AI-driven proof generation tool to automatically synthesize the Lean 4 proof script connecting these SVA assumptions to the final IEEE-754 specification. This formalization step also serves as a feedback loop, exposing any subtle gaps in our conceptualized proof and prompting the refinement or addition of SVA properties.

II. Background

As noted in the introduction, we illustrate our methodology through the verification of a particular hardware design that

implements a radix-16 SRT square root algorithm. By way of background, we briefly rehearse the key components of the algorithm here, as presented in [12].

Given an input radicand $X \in [\frac{1}{4}, 1)$, the SRT algorithm chooses a sequence of *root digits* $q_j \in \{-a, \ldots, a\}$ where a is an integer with $\frac{1}{2}(r-1) < a \leq r-1$. Here r is the *radix*, in our case, 16. Correspondingly, a sequence of *partial roots* $Q_0, Q_1, \ldots$ and *partial remainders* $R_0, R_1, \ldots$ are constructed satisfying the following recurrence relations:

$$\begin{aligned} Q_0 &= 1 \\ Q_{j+1} &= Q_j + r^{-(j+1)} q_{j+1} \\ R_0 &= X - 1 \\ R_{j+1} &= rR_j - q_{j+1}(2Q_j + r^{-(j+1)} q_{j+1}^2) \end{aligned}$$

It can be proven, by induction using these recurrences, that:

$$R_j = r^j (X - Q_j^2). \tag{1}$$

The root digits q_j are selected with the goal of preserving the invariant

$$-2\rho Q_j + \rho^2 r^{-j} \leq R_j \leq 2\rho Q_j + \rho^2 r^{-j}, \tag{2}$$

where $\rho = \frac{a}{r-1}$ is the *redundancy factor*. Substituting (1) into (2) and rearranging, we get

$$|Q_j - \sqrt{X}| \leq \rho r^{-j}. \tag{3}$$

Hence, (2) guarantees that the partial roots converge to the true square root of the radicand X.

Although the choice of root digits is clearly important for the algorithm, the details turn out not to be important for our example verification, so we do not dwell on them here.

III. Invariant Discovery

A common challenge in hardware verification is that implementations rarely match textbook theory one-to-one. Textbook presentations often assume infinite-precision real numbers and unbounded iterations, whereas hardware implementations employ fixed-width integer registers, implicit bit shifts, and fixed numbers of cycles. In this section, we present the workflow used to bridge this gap, detailing how textbook algorithms are, in our methodology, refined in light of counterexamples.

A. Proving Recurrence Relations

In [12], R_j and Q_j are treated as rational numbers. In physical hardware, however, signals are represented as unsigned or two's complement integers with implicit binary points. Analyzing traces allows us to determine the exact binary point offsets and define signals Q and R as bit shifts of RTL signals such that the implicit binary points are aligned. An iterative process of attempting proofs and analyzing counterexample traces results in the following recurrence relations

$$\begin{aligned} Q_0 &= 0, \qquad & Q_{j+1} &= Q_j + 2^{54-4j} q_j, \\ R_0 &= X, \qquad & R_{j+1} &= 16R_j - 8Q_j q_j - 16^{-j} q_j^2, \end{aligned} \tag{4}$$

where X is the input operand scaled so that $2^{56} \leq X < 2^{58}$.

B. Counterexample-Guided Bound Relaxation

Inspecting the RTL reveals that q_j ranges over $\{-9, \ldots, 9\}$, implying that $a = 9$. The textbook formula for the redundancy factor tells us that $\rho = 0.6$. But simply asserting $\rho = 0.6$ in (2) yields counterexamples from the model checker. And textbook theoretical bounds tell us that we should have $\frac{1}{2} < \rho \leq 1$, but even setting $\rho = 1$ results in counterexamples.

Our solution is to analyze the inequality (3) and find the maximum value of ρ resulting in a suitable degree of accuracy for the final partial root. An inspection of execution traces and RTL code shows us that the RTL operates over a maximum of 15 iterations ($j = 0, \ldots, 14$) with early termination if $R_j = 0$. To achieve the required precision for a correctly rounded 64-bit floating-point result, we must therefore have $|Q_{14} - \sqrt{X}| < 2^{-54}$, [1] so any value of ρ less than 4 in (3) would be appropriate. Setting $\rho = 3$ produces the following hardware-verified invariant inequality:

$$9 \cdot 2^{56-4j} - 6 \cdot Q_j \leq R_j \leq 9 \cdot 2^{56-4j} + 6 \cdot Q_j \tag{5}$$

The critical remaining step is to verify whether this weaker invariant (5) together with (4) still guarantees sufficient precision for correctness.

C. Closed-Form Deduction and Precision Sufficiency

We deduce from (4) that

$$2^{56-4j} R_j = 2^{56} X - Q_j^2. \tag{6}$$

Combining this with (5) and writing $S = 2^{56} X$ we find that

$$(Q_j - 3 \cdot 2^{56-4j})^2 \leq S \leq (Q_j + 3 \cdot 2^{56-4j})^2.$$

That is,

$$|Q_j - \sqrt{S}| \leq 3 \cdot 2^{56-4j}$$

So the partial roots Q_j become closer to the square root of the scaled operand S on each iteration and in the final iteration we have that

$$|Q_{14} - \sqrt{S}| \leq 3 \tag{7}$$

Double-precision floats have 52 mantissa bits, so including the implicit bit, our final result should be correct to 53 significant bits. Since $2^{56} \leq X < 2^{58}$, we have $1 \leq 2^{-56}\sqrt{S} < 2$. Scaling (7) we obtain $|2^{-56} Q_{14} - 2^{-56}\sqrt{S}| \leq 3 \cdot 2^{-56} < 2^{-54}$. This guarantees that, with correct rounding in place, the final result will be correct to the required 53 significant bits.

IV. Proof Conceptualization and Lemma Verification

Establishing the core algorithmic invariants confirms that the iterative core of the circuit correctly computes the partial roots and remainders. But this alone does not guarantee end-to-end correctness. A complete proof must also establish that the unit's final output registers contain a correct result upon the unit signalling completion.

[1] An upper bound of 2^{-54} guarantees that the distance between Q_{14} and the true mathematical value $\sqrt{X}$ is strictly less than half a Unit in the Last Place (ULP) of the 53-bit mantissa, the precision necessary for a correctly rounded result.

It is clear, of course, that additional properties of the RTL are required. But rather than simply proving ad-hoc assertions that we think are missing, in our methodology we begin by formulating a high-level proof skeleton informed by the architectural design. This guides us towards the intermediate lemmas required to link the iterative core to the final output registers.

A. RTL inspection

Inspecting the RTL reveals that the circuit is partitioned into four pipeline stages:

1) **Input Preparation:** Extracts the sign, exponent, and normalized mantissa from the stored operand and detects special cases.
2) **SRT:** Executes the iterative digit-recurrence algorithm over a maximum of 15 iterations.
3) **Rounding:** Analyzes the final partial root Q, remainder R, and rounding mode `rm_q` to adjust the mantissa.
4) **Packing:** Computes the final exponent and assembles the sign, exponent, and mantissa fields into the output register `unit_result`.

This architectural design suggests a natural proof strategy: prove lemmas characterizing each individual stage and prove safe handling of inputs and outputs between stages.

B. Proof Conceptualization

As discussed in section III, in our example verification we prove (4) and (5), although the specific SVAs don't match these equations one-to-one. In particular, we have the following:

$$\texttt{Q_initial: first} \to Q = 0$$
$$\texttt{R_initial: first} \to R = X$$
$$\texttt{Q_recur: (first} \vee \texttt{middle)} \to Q_{\text{next}} = Q + 2^{54-4j}q$$
$$\texttt{R_recur: (first} \vee \texttt{middle)} \to R_{\text{next}} = 16R - 8Qq - 16^{-j}q^2$$
$$\texttt{Inv: (middle} \vee \texttt{last)} \to 9 \cdot 2^{56-4j} - 6 \cdot Q \leq R \leq 9 \cdot 2^{56-4j} + 6 \cdot Q$$

The signals `first`, `middle`, and `last` mark the first cycle, last cycle, or an in-between cycle of the SRT stage. At this stage we only suspect that these signals behave this way by analyzing traces, rather than formally proving properties. Similarly, we don't yet prove that j really marks the number of cycles since the initial iteration.

Ultimately, we want to prove (7). As seen in section III, we first need to prove (6), which requires a mathematical induction. In particular, we want to show

$$(\texttt{first} \vee \texttt{middle} \vee \texttt{last}) \to 2^{56-4j}R = 2^{56}X - Q^2, \quad (8)$$

by induction on j.

In order to derive targeted lemmas that will allow us to complete this mathematical induction in Lean 4, we sketch a mathematical proof of (8) by hand. To prove the base case ($j = 0$) in Lean 4, we will need to utilize `Q_initial` and `R_initial`, so we prove

$$((\texttt{first} \vee \texttt{middle} \vee \texttt{last}) \wedge j = 0) \to \texttt{first}.$$

Additional lemmas are needed for the inductive step. We will not exhaustively list these here, but only isolate a selection to illustrate the thought process.

It is clear that `R_recur` and `Q_recur` are necessary for the inductive step. In order to link R_{next} with R and Q_{next} with Q, we must prove:

$$(\texttt{middle} \vee \texttt{last}) \to R = \texttt{\$past}(R_{\text{next}})$$
$$(\texttt{middle} \vee \texttt{last}) \to Q = \texttt{\$past}(Q_{\text{next}}),$$

where `$past(x)` is an operator returning the value of `x` in the previous clock cycle. However, this alone would not be enough to use `R_recur` and `Q_recur`, so we must also prove

$$(\texttt{middle} \vee \texttt{last}) \to \texttt{\$past}(\texttt{first} \vee \texttt{middle}).$$

Additional properties involving j and the stability of X are also required, although we do not list these here.

Guided by the conceptual (pencil and paper) mathematical proof sketched in this step, we continue proving assertions in this way until we believe we have all the necessary lemmas to formally prove, within Lean 4, that the input of the rounding module satisfies (7).

C. RTL-guided Lemmas

Having dealt with the SRT stage (skipping over the input preparation stage), we must now prove properties about the rounding and packing stages. Since these stages should involve routine case analyses with a relatively small amount of logic involved compared to the SRT stage, the approach is to inspect the RTL and write SVAs that simplify the more complex bit manipulation in the RTL. The result is a collection of SVAs that summarize the roles of the rounding and packing stages. This sets us up to complete the proof in Lean 4.

V. Transfer to an ITP and Formal Proof Completion

The final phase of our methodology transfers SVA properties into an ITP to establish top-level compliance with the IEEE-754 specification. At this point we should ask ourselves why we should use a theorem prover at all, rather than sticking to model checking for the entire proof. One obvious point is that the IEEE-754 specification is a statement about real numbers, which clearly cannot be written directly as an SVA. However, with some mathematical manipulation, one may formulate an equivalent bit-level statement expressed as an SVA and content themselves with proving that. Another, more significant, point is that model checkers are simply not effective at handling non-linear expressions involving multiplication, struggling to handle the large problem scale and complexity.

In our proof development for this example, we tested this by attempting to use model checking to prove (6), a crucial equality for the verification. We gave the model-checking software all the relevant proven SVAs as assumptions, utilised 'stopats' which treated the signals appearing in the SVA as free variables, and used state-of-the-art word-level engines. Despite these efforts, the model checker failed to provide any counterexamples or prove the assertion and timed out after 24 hours.

Switching to a theorem prover, we abstract the RTL as a collection of variables in Lean 4, express the top-level theorem as mathematical inequalities over real numbers, and complete the formal proof using a combination of AI-assisted proof generation and SVA refinement.

A. State Abstraction and Signal Modeling

We first define a Lean 4 inductive type (which can be thought of as a struct), `Signal`, that comprises a minimal set of signal names extracted from our SVA lemmas. Appropriate types, such as `Bool` or `BitVec 64`, are assigned to each signal name through the use of a Lean 4 function type `SignalVal : Signal → Type`.

In order to simplify the proofs in Lean 4, we model signed bit vector signals as unbounded integers ($\mathbb{Z}$). We justify this treatment by proving SVA properties that show that overflow and underflow never occur in the arithmetic involved in our SVA lemmas. An alternative approach would be to use bit vectors alongside helper functions to handle the signed arithmetic.

We then abstract the state of the RTL as a Lean 4 dependent type `State := ∀ s : Signal, SignalVal s`. Essentially, `State` assigns each signal name `s` listed in `Signal` to a value with the corresponding type `SignalVal s`. This allows us to abstract an execution trace `tr : ℕ → State` as a function type, taking in a time `t : ℕ` and outputting a state.

In our proof development, the translation of our SVA lemmas into Lean 4 hypotheses was primarily automated using an agentic AI coding assistant (GitHub Copilot [8]), followed by manual verification and correction. It should be noted that this approach is potentially prone to human or AI errors. A potential avenue for further development of this methodology is writing a verified SVA to Lean 4 compiler, which would reduce the potential for error.

B. Top-Level IEEE-754 Specification

The top-level theorem says that `unitResult` is the correctly rounded double-precision floating-point square root of the stored operand `srcf0Q`. Mathematical correctness can be expressed by an inequality, depending on the chosen rounding mode `rmQ`, between $\texttt{fp64ToReal}(\texttt{unitResult})$ and $\sqrt{\texttt{fp64ToReal}(\texttt{srcf0Q}_t)}$. In particular, the theorem says:

Theorem 1 (`sqrtCorrect`)**.** Let tr satisfy `ValidTrace` and let t be a cycle where $\texttt{unitDone}_t$, $\texttt{noSpecial}_t$, and $\texttt{sqrt64B}_t$ are all `true`.

Write $res = \texttt{unitResult}_t$, let $y = \texttt{fp64ToReal}(res)$, let y_+, y_- be its neighbors one ulp above and below, let $y_{+\frac{1}{2}}, y_{-\frac{1}{2}}$ be the midpoints half an ulp above and below, and let $\sigma = \sqrt{\texttt{fp64ToReal}(\texttt{srcf0Q}_t)}$. Then, with $rm = \texttt{rmQ}_t$:

$$rm \in \{\mathrm{RTZ}, \mathrm{RDN}\} \implies y \le \sigma < y_+,$$
$$rm = \mathrm{RUP} \implies y_- < \sigma \le y,$$
$$rm = \mathrm{RNE} \implies \big(y_{-\frac{1}{2}} < \sigma < y_{+\frac{1}{2}}\big) \vee \big(\sigma \in \{y_{-\frac{1}{2}}, y_{+\frac{1}{2}}\} \wedge res \text{ has even significand}\big),$$
$$rm = \mathrm{RMM} \implies y_{-\frac{1}{2}} \le \sigma < y_{+\frac{1}{2}}.$$

In theorem 1, `ValidTrace` is a property of a trace $tr : \mathbb{N} \to$ `State` that says *tr* satisfies the conjunction of all the translated SVA lemmas. `unitDone` signalled completion of the DivSqrt unit and `noSpecial` and `sqrt64B` are flags defined in Verilog that say that the operation is non-special (not negative, NaN, infinite, or zero) and a 64-bit square root operation, respectively. Special case operations may be straightforwardly formally verified with a separate SVA, according to the IEEE-754 standard.

C. Proof Completion

In our proof development for this example of our methodology, our exploratory approach to theorem proving resulted in two primary stages:

1) We first proved (8) by induction on j. Formalizing the proof in Lean 4 exposed gaps in the initial SVA lemma formulation, such as an explicit bound on $j \in [0, 14]$. These missing lemmas were fed back to the model checker to be verified on the low-level RTL.
2) In order to reduce the time required to complete the proof, we utilized Harmonic's Aristotle AI proof engine [3] to generate a proof of Theorem 1 with a 'fill in the sorries' prompt. The automated tool completed the proof within approximately two hours. But along the way, it highlighted gaps in the set of assumptions which it filled in with provisional assumptions. This prompted us to refine our SVA lemmas and complete the proof.

VI. Related Work

The combination of hardware verification by model checking with deductive theorem proving has a long history. The central motivation is the same as in this work. Model checking and symbolic simulation are effective at establishing concrete properties of a design, and can often deal well with microarchitectural complexity, while theorem provers can express and reason about higher-level statements of correctness. This reasoning can range from simple 'composition' of model-checked properties to more mathematical proofs, such as IEEE-754 compliance—which requires reasoning about the reals.

The HOL-Voss system [5] was an early example of this combination, embedding model checking within a general-purpose theorem prover. Its symbolic trajectory evaluation [7] engine could efficiently establish properties of datapath-oriented circuits, while the surrounding higher-order logic allowed these results to be composed to obtain higher level results.

Aagaard, Jones, and Seger developed this idea further with Lifted-FL [1]. This approach integrated symbolic trajectory evaluation with theorem proving in a common functional language, allowing model-checking results to be lifted into the logic and used in deductive proofs. The important contribution was not merely the availability of both techniques, but their use within a single verification environment. The general style of proof supported by this integration resembles the methodology used here. Model-checking engines were used to establish low-level facts about a concrete implementation, which then form the basis for a higher-level mathematical argument.

The Forte environment [13] demonstrated that this combination could support industrial-scale verification. Forte coupled symbolic trajectory evaluation with lightweight higher-order theorem proving and a programmable verification language. Its methodology emphasized decomposing a large proof into tractable model-checking problems and using theorem proving to connect the resulting properties. The work of Seidel [14] similarly argues for a combination of theorem proving and model checking tools in functional verification.

Most recently [10, 9], Lean 4 was used in conjunction with model checking in establishing a proof of specification compliance for the open source Ibex RISC-V core, where properties sufficient for correctness were expressed in SystemVerilog Assertions and proved by model checker, before being translated as axioms to Lean 4. From there those axioms could be used to establish the high level correctness property of Ibex by hand.

These approaches establish that the division of labor between automatic and deductive methods is practical and useful. But they still require the user to construct the deductive proof that relates the low-level properties to the top-level specification.

Our work follows the same broad division, but stages the interaction somewhat differently. We first use the mathematical structure of the SRT algorithm and the RTL microarchitecture to formulate a proof conceptually. This proof concept guides the discovery of a collection of targeted SVA lemmas, each of which can be checked directly against the RTL. We then transfer the resulting facts to Lean 4, where the remaining argument is expressed over integers and reals rather than over bit vectors. The theorem prover is used at the point where the proof becomes principally mathematical: establishing an inductive closed form for the recurrence, deriving a precision bound, and connecting that bound to the IEEE-754 rounding specification.

A contrasting approach is to carry out the full verification within a theorem prover. Russinoff's verification of the AMD-K7 floating-point multiplication, division, and square-root algorithms in ACL2 is the canonical example [11]. That work formalized bit-vector representations, floating-point formats, rounding, the hardware-level algorithm, and the final IEEE-754 correctness statements in ACL2. It thereby produced a fully mechanically checked proof spanning the semantic gap between RTL-level operations and numerical correctness.

This style of verification has the advantage of a uniform logical setting and can achieve a very strong end-to-end result. At the same time, it requires considerable theorem-proving effort, including the development of detailed hardware models and a large body of supporting arithmetic lemmas. Russinoff notes both the investment involved and the importance of minimizing manual translations between design representations and the theorem-prover model.

The present methodology occupies a middle ground. Direct model checking is used where it is strongest: proving cycle-accurate invariants and interface properties of the original RTL, with counterexamples helping to refine incorrect or incomplete candidate assertions. Lean is then used only for the relatively small abstraction of the RTL that is needed to state and prove IEEE-754 conformance.

The final difference from earlier hybrid methodologies is the use of an AI-driven Lean proof tool to synthesize much of this mathematical proof. Systems such as LeanDojo [16] and DeepSeek-Prover-V1.5 [15] show that proof-assistant feedback can guide language-model-based proof search effectively. For our work, we simply used Harmonic's Aristotle tool [3].

In our setting, the generated proof is still checked by Lean. The AI is a proof-engineering aid rather than an additional source of trust. More importantly, unsuccessful proof attempts expose missing assumptions in the proposed lemma set, providing a feedback path from the high-level proof back to RTL-level model checking. This makes the methodology intended not only to reduce manual proof effort, but also to make gaps in the informal correctness argument easier to detect.

VII. Conclusion

In this paper, we have presented a hybrid methodology for the formal verification of floating-point hardware designs. The methodology is founded on earlier work such as the HOL-Voss system [5], Lifted-FL [1], and the Forte environment [13], which combine model checking for low-level RTL logic and deductive theorem proving for high-level proof synthesis. Our modern revision of this approach utilises AI-assisted proof generation for the high-level theorem proving to significantly reduce the human proof engineering overhead.

Our approach begins with targeted SVA lemma verification by model checking. To guarantee effective lemma formulation, we have highlighted the importance of developing a conceptual proof, guided by the mathematical algorithm implemented and the specific RTL design. The next stage involves 'composing' these lemmas through the use of AI-assisted interactive theorem proving. This stage highlights potential gaps in the lemmas, allowing for a feedback loop of SVA refinement and high-level proof synthesis, culminating in a top-level correctness result. We have demonstrated the effectiveness of our methodology by verifying an open source 64-bit SRT square root hardware implementation [6] against the IEEE-754 floating point standard [4].

A potential gap in our current approach is that the translation of SVA into Lean and the RTL abstraction could be prone to error. Future efforts to further develop this methodology could involve the creation of a formally verified SVA to Lean compiler to mechanically automate this translation process.

References

[1] Mark D. Aagaard, Robert B. Jones, and Carl-Johan H. Seger. "Lifted-FL: A Pragmatic Implementation of Combined Model Checking and Theorem Proving". In: *Theorem Proving in Higher Order Logics*. Springer, 1999.

[2] Ashraf Alam and Louis-Emile Ploix. *Proof of Fpnew floating point square root*. 2026. URL: https://github.com/OX-FV/fpnew-sqrt-formal.

[3] Harmonic. *Aristotle*. 2026. URL: https://aristotle.harmonic.fun/.

[4] IEEE. *754-2019 - IEEE Standard for Floating-Point Arithmetic*. July 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[5] Jeffrey J. Joyce and Carl-Johan H. Seger. "The HOL-Voss System: Model-Checking inside a General-Purpose Theorem-Prover". In: *Higher Order Logic Theorem Proving and its Applications, 6th International Workshop, HUG '93, Vancouver, BC, Canada, August 11-13, 1993, Proceedings*. Ed. by Jeffrey J. Joyce and Carl-Johan H. Seger. Vol. 780. Lecture Notes in Computer Science. Springer, 1993, pp. 185–198.

[6] Stefan Mach et al. "FPnew: An Open-Source Multiformat Floating-Point Unit Architecture for Energy-Proportional Transprecision Computing". In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 29.4 (Dec. 2020), pp. 774–787. DOI: 10.1109/TVLSI.2020.3044752.

[7] Tom Melham. "Symbolic Trajectory Evaluation". In: *Handbook of Model Checking*. Ed. by Edmund M. Clarke et al. Springer International Publishing, 2018. Chap. 25, pp. 831–870. ISBN: 978-3-319-10574-1. DOI: 10.1007/978-3-319-10575-8_25.

[8] Microsoft. *Github Copilot*. 2026. URL: https://github.com/features/copilot.

[9] Louis-Emile Ploix. *Ibex: Lean proof of Observational Correctness*. 2025. URL: https://github.com/mndstrmr/OS-formal-ibex/commit/db6cb233302bc277f7bada8e0e57cec609ecd087.

[10] Louis-Emile Ploix et al. *Comprehensive Formal Verification of Observational Correctness for the CHERIoT-Ibex Processor*. 2025. arXiv: 2502.04738 [cs.AR]. URL: https://arxiv.org/abs/2502.04738.

[11] David M. Russinoff. "A Mechanically Checked Proof of IEEE Compliance of the Floating Point Multiplication, Division and Square Root Algorithms of the AMD-K7 Processor". In: vol. 1. 1998, pp. 148–200.

[12] David M. Russinoff. *Formal Verification of Floating-Point Hardware Design*. Springer, 2019.

[13] Carl-Johan H. Seger et al. "An Industrially Effective Environment for Formal Hardware Verification". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 24.9 (2005), pp. 1381–1405.

[14] Peter-Michael Seidel. "A Case for Multi-level Combination of Theorem Proving and Model Checking Tools". In: *2014 15th International Microprocessor Test and Verification Workshop*. 2014, pp. 90–97.

[15] Huajian Xin et al. "DeepSeek-Prover-V1.5: Harnessing Proof Assistant Feedback for Reinforcement Learning and Monte-Carlo Tree Search". In: *International Conference on Learning Representations*. 2025.

[16] Kaiyu Yang et al. "LeanDojo: Theorem Proving with Retrieval-Augmented Language Models". In: *Advances in Neural Information Processing Systems*. 2023.